

第六章 输入/输出流和文件

目录

- 6.1 输入/输出流
- 6.2 文件读写
- 6.3 本章小结

6.1 输入/输出流

- 通常程序需要从外部获取/输出信息
 - 这个“外部”范围很广，包括诸如键盘、显示器、文件、磁盘、网络、另外一个程序等
 - “信息”也可以是什么类型的，例如一个对象、字符串、图像、声音等
- 通过使用java.io包中的输入/输出流类就可以达到输入输出信息的目的

6.1.1 I/O流的概念

- I/O流(Input/Output)

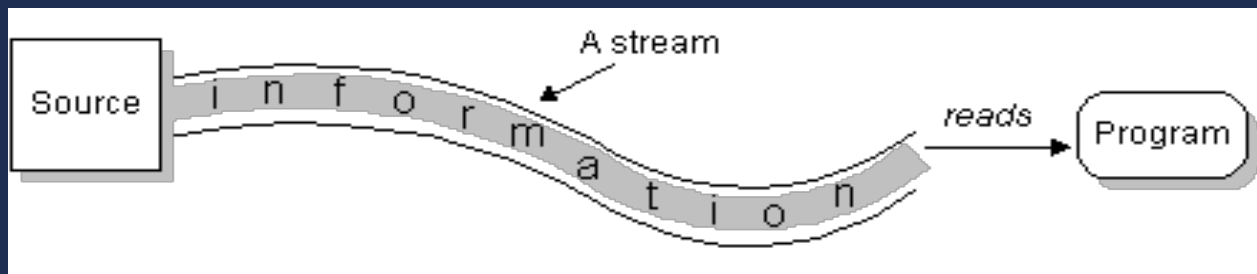
- 在Java中将信息的输入与输出过程抽象为I/O流
 - 输入是指数据流入程序
 - 输出是指数据从程序流出
- 一个流就是一个从源流向目的地的数据序列
- IO流类一旦被创建就会自动打开
- 通过调用close方法，可以显式关闭任何一个流，如果流对象不再被引用，Java的垃圾回收机制也会隐式地关闭它

输入
输出流

6.1.1 I/O流的概念(续)

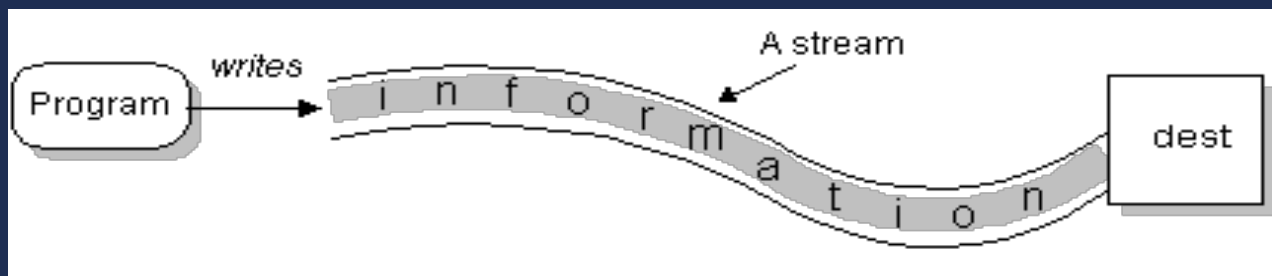
● 输入流

- 为了从信息源获取信息，程序打开一个输入流，程序可从输入流读取信息



● 输出流

- 当程序需要向目标位置写信息时，便需要打开一个输出流，程序通过输出流向这个目标位置写信息



6.1.1 I/O流的概念(续)

——源和目标的类型

输入
输出流

对象	源? 目标? 或两者?
disk file	Both
running program	Both
monitor	Destination
keyboard	Source
Internet connection	Both
image scanner	Source
mouse	Source

6.1.1 I/O流的概念(续)

——读写数据的方法

- 不论数据从哪来，到哪去，也不论数据本身是何类型，读写数据的方法大体上都是一样的：

读	写
打开一个流 读信息 关闭流	打开一个流 写信息 关闭流

输入/输出流

6.1.2 预定义的I/O流类概述

输入/输出流

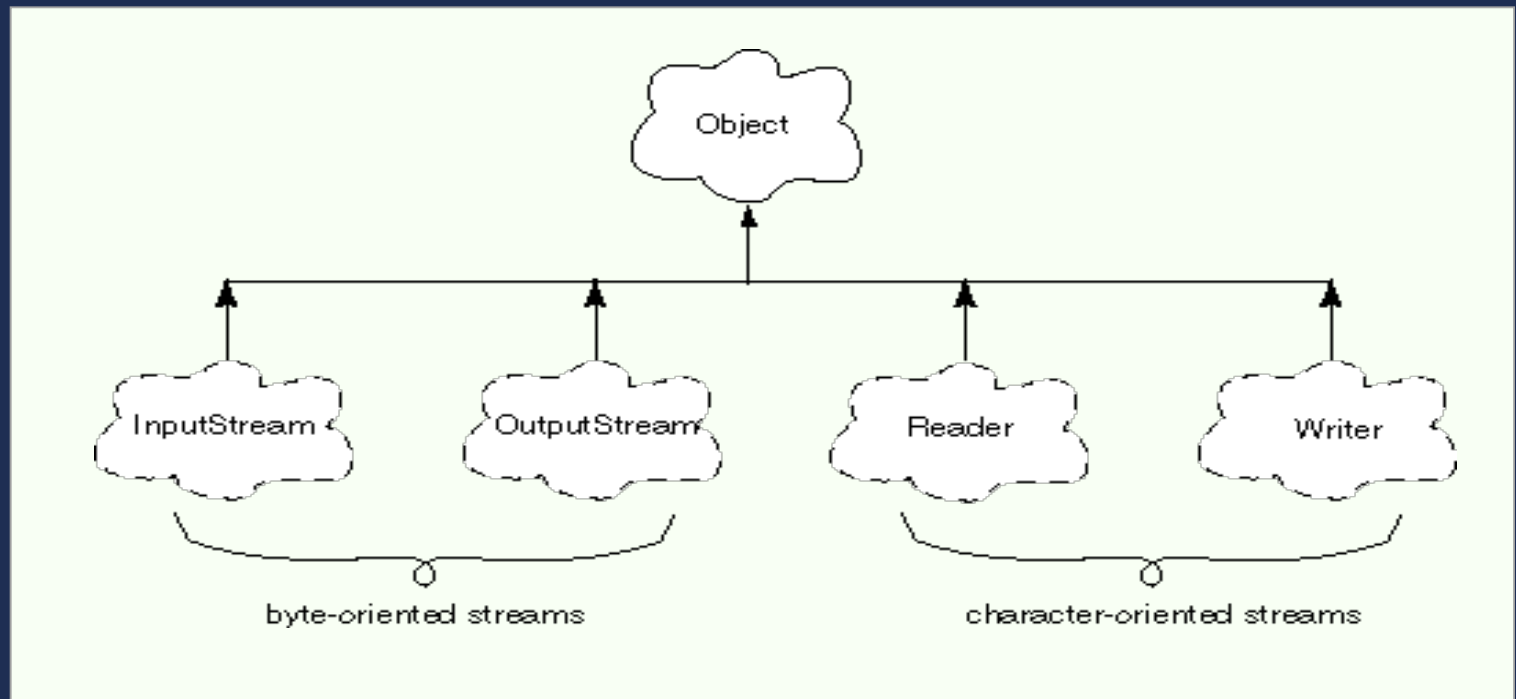
- 输入/输出流可以从以下几个方面进行分类
 - 从流的方向划分
 - 输入流
 - 输出流
 - 从流的分工划分
 - 节点流
 - 处理流
 - 从流的内容划分
 - 面向字符的流
 - 面向字节的流

6.1.2 预定义的I/O流类概述(续)

——java.io包的顶级层次结构

- 面向字符的流：专门用于字符数据
- 面向字节的流：用于一般目的

输入/输出流



6.1.2 预定义的I/O流类概述(续)

——java.io包的顶级层次结构

输入
输出流

- 面向字符的流：
 - 专门用于字符数据
 - 源或目标通常是文本文件
 - Java中的字符使用16-bit 的Unicode编码，每个字符占两个字节
- 面向字节的流：
 - 用于一般目的

6.1.2 预定义的I/O流类概述(续)

——面向字符的流

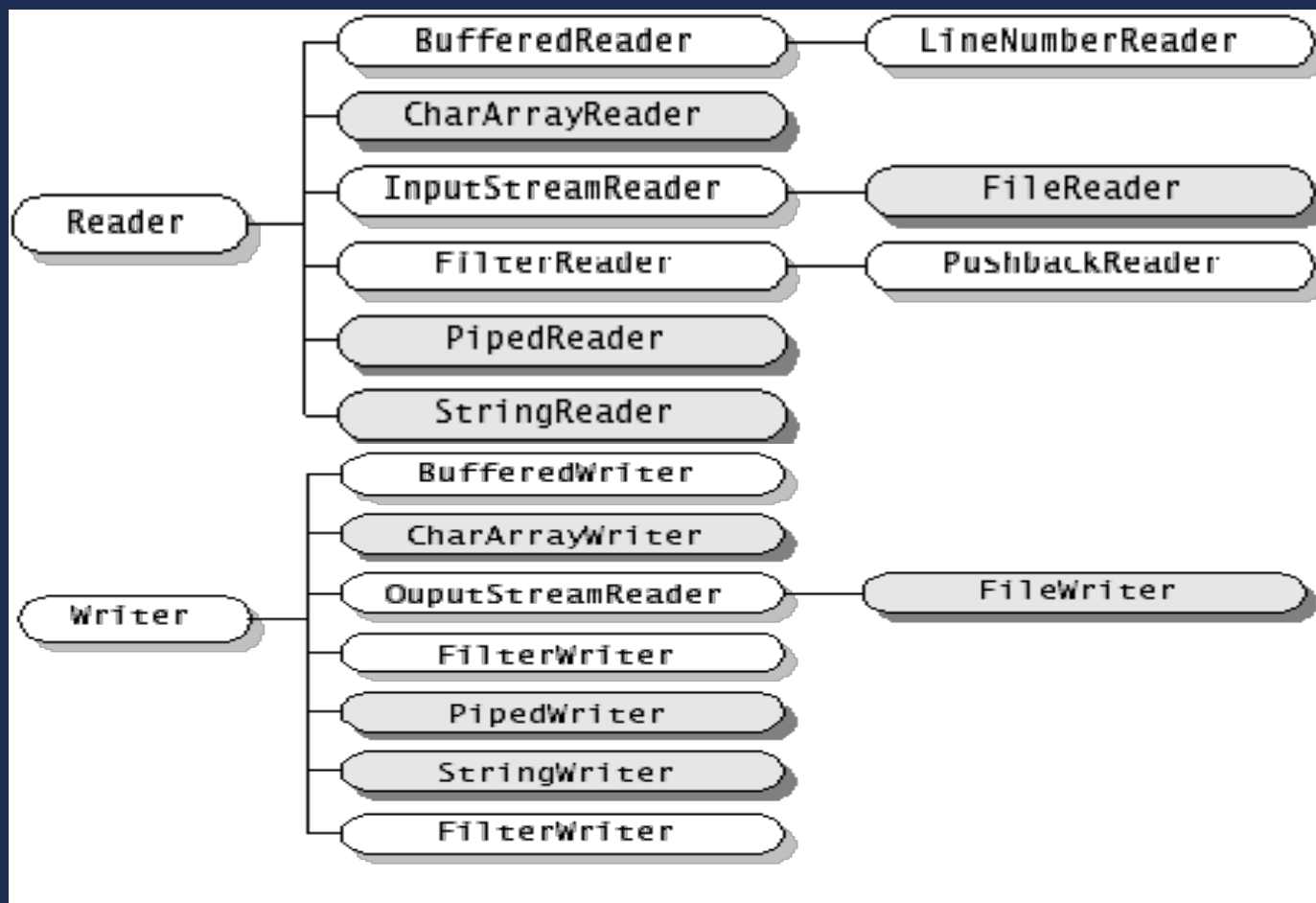
输入
输出流

- 面向字符的抽象类——**Reader**和**Writer**
 - **java.io**包中所有字符流的抽象基类
 - **Reader**提供了输入字符的API
 - **Writer**提供了输出字符的API
 - 它们的子类又可分为两大类
 - 节点流：从数据源读入数据或往目的地写出数据
 - 处理流：不直接与数据源或目标相连，而与另外的流进行配合
 - 如：**BufferedReader**对另一个流产生的数据进行缓冲
 - 多数程序使用这两个抽象类的一系列子类来读入/写出文本信息
 - 例如**FileReader/FileWriter**用来读/写文本文件

6.1.2 预定义的I/O流类概述(续)

——面向字符的流

输入/输出流



阴影部分为节点流

6.1.2 预定义的I/O流类概述(续)

——面向字节的流

输入
输出流

- 数据源或目标中含有非字符数据，必须用字节流来输入/输出
- 通常被用来读写诸如图片、声音之类的数据(二进制)
- 绝大多数数据是被存储为二进制文件的，世界上的文本文件大约只能占到2%，通常二进制文件要比含有相同数据量的文本文件小得多

6.1.2 预定义的I/O流类概述(续)

——面向字节的流

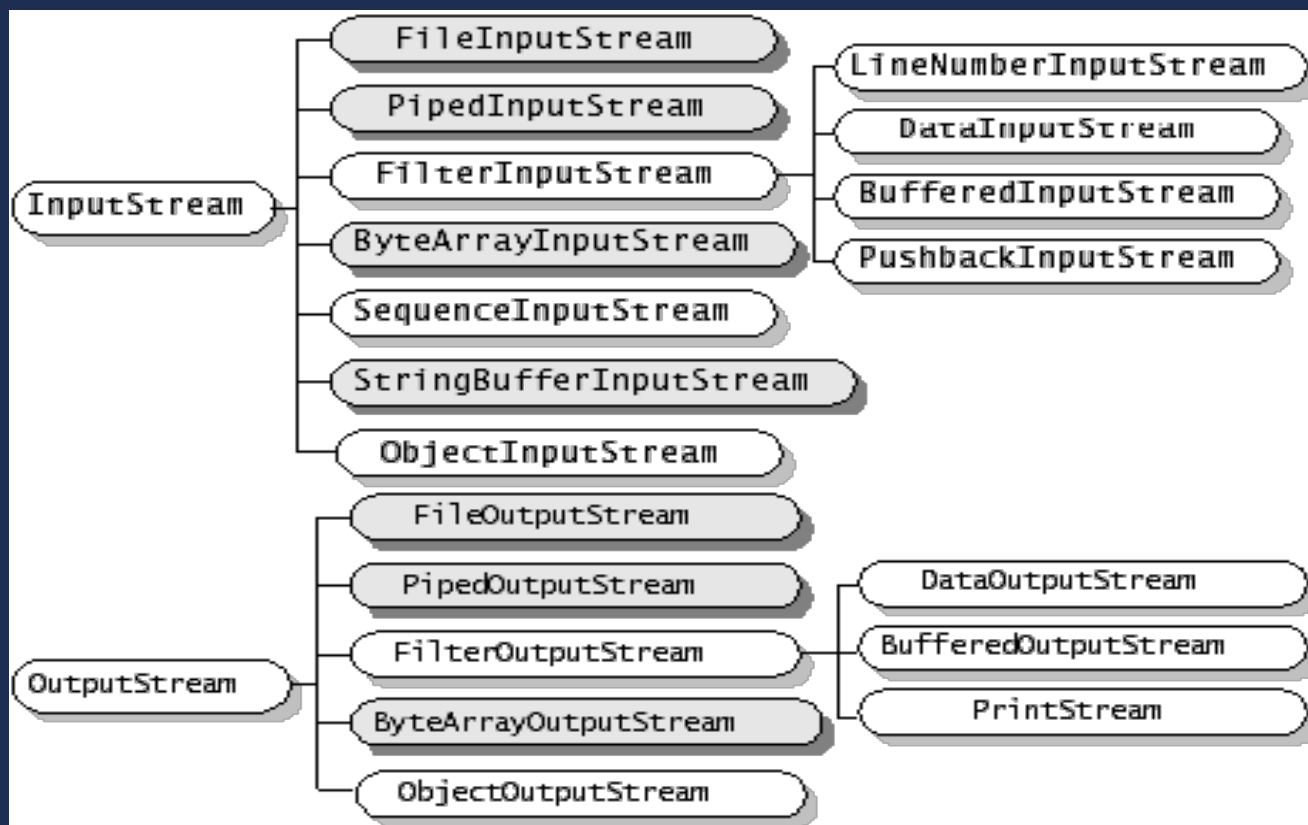
输入
输出流

- **InputStream和OutputStream**
 - 是用来处理8位字节流的抽象基类，程序使用这两个类的子类来读写8位的字节信息
 - 分为两部分
 - 节点流
 - 处理流

6.1.2 预定义的I/O流类概述(续)

——面向字节的流

输入/输出流



阴影部分为节点流

6.1.2 预定义的I/O流类概述(续)

——标准输入输出

输入
输出流

- 标准输入输出流对象
 - **System**类的静态成员变量
 - 包括
 - **System.in**: **InputStream**类型的, 代表标准输入流, 这个流是已经打开了的, 默认状态对应于键盘输入。
 - **System.out**: **PrintStream**类型的, 代表标准输出流, 默认状态对应于屏幕输出
 - **System.err**: **PrintStream**类型的, 代表标准错误信息输出流, 默认状态对应于屏幕输出

6.1.2 预定义的I/O流类概述(续)

——例6_1

- 从键盘读入信息并在显示器上显示

```
import java.io.*;

public class Echo {
    public static void main(String[] args) throws IOException {
        BufferedReader in = new BufferedReader(
            new InputStreamReader(System.in));

        String s;
        while((s = in.readLine()).length() != 0)
            System.out.println(s);
    }
}
```

运行结果

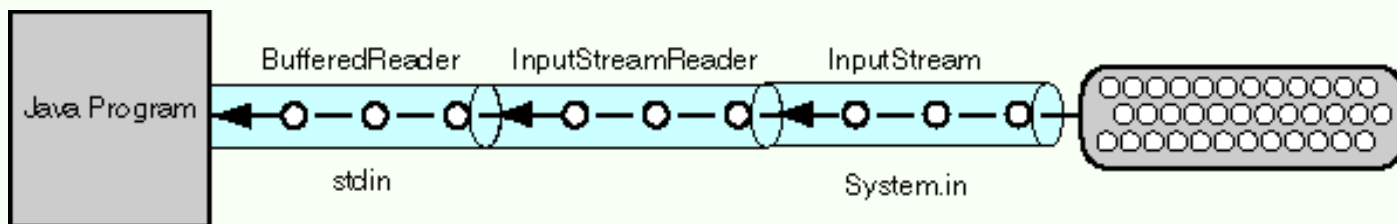
Hello!

Hello!

6.1.2 预定义的I/O流类概述(续)

——处理流

输入/输出流



用一行表达式实现：

```
BufferedReader stdin = new BufferedReader  
    ( new InputStreamReader(System.in) );
```

6.1.2 预定义的I/O流类概述(续)

——I/O异常

输入
输出流

- IO异常
 - 多数IO方法在遇到错误时会抛出异常，因此调用这些方法时必须
 - 在方法头声明抛出IOException异常
 - 或者在try块中执行IO，然后捕获IOException

6.2 文件读写

- **File**类
- 写文本文件
- 读文本文件
- 写二进制文件
- 读二进制文件
- 处理压缩文件
- 对象序列化
- 随机文件读写

1. File类

- 表示磁盘文件信息
- 定义了一些与平台无关的方法来操纵文件
 - 创建、删除文件
 - 重命名文件
 - 判断文件的读写权限及是否存在
 - 设置和查询文件的最近修改时间等
- 构造文件流可以使用**File**类的对象作为参数

- **File class is useful for obtaining information about files and directories.**
- **Constructor for File**
 - **File(String pathname)**
 - `File file = new File("D:\\Java\\test.Java");`
 - **File(String pathTname, String name)**
 - `File file = new File("D:\\Java\\", "test.Java");`
 - **File(File dir, String name)**
 - `File dir=new File("D:\\Java\\");`
 - `File file = new File(dir, "test.Java");`
 - Use the previously created File object directory

构造方法

kp] gd^ Adg` #Nbnali b i \h` \$	赘帜蔽 Adg` 着□哄□□根锄塘□嘭□另□ i \h` 料 抽俊颇□岍端阖联根锄塘□嘭另
kp] gd^ Adg` #Nbnali b k\ocQ I \h` ' Nbnali b i \h` \$	登□立竦 k\ocQ I \h` ° □着□岍塘□着□岍 ` 鲈帜 大立竦 i \h` 栝赘帜□根锄塘□嘭
kp] gd^ Adg` #Adg` _dm^oj m' Nbnali b i \h` \$	登□□蹕□ Adg` 着□ _dm^oj m # □着□岍塘□着□ 岍\$鲈帜大立竦 i \h` 栝赘帜□根锄塘□嘭

常用方法

ljjg` \i ^\i M_#\$	朽鳝根锄料□ !枷□晌□ !簌枷□晌酚
ljjg` \i ^\i Rnlo`#\$	朽鳝根锄料画 !枷□晌□ !簌枷□晌酚
ljjg` \i `sdnon#\$	朽鳝 Adg` 鲶□汲竦立竦栳赘帜□另□蟾赘帜□岍表 □根锄瑯□嘭 !枷□晌□ !簌枷□晌酚
ljjg` \i ^m\o`l`rAdg`#\$	朽鳝根锄蓖锳屎 !枷佳咣□标另者□□根锄 !薜□ 晌□ !朽鳝根锄锳屎 !枷□晌酚
ljjg` \i dnAdg`#\$	朽鳝 Adg` 鲶□汲竦立竦栳赘帜□另□蟾崩标根锄 ! 枷□晌□ !簌枷□晌酚
ljjg` \i dn?dm^oj m#\$	朽鳝 Adg` 鲶□汲竦立竦栳赘帜□另□蟾崩标□嘭 ! 枷□晌□ !簌枷□晌酚
ljjg` \i dn<] nj gpo`#\$	朽鳝 Adg` 鲶□汲竦立竦栳赘帜□另□蟾崩标根锄瑯 □嘭□□着□岍 !枷□晌□ !簌枷□晌酚
ljjg` \i -`g`o`#\$	贾□根锄瑯□嘭 !朽鳝蟾□嘭 !彡□蟾□□嘭栲□ 贾□瑁谨 !贾□瑁谨□晌□ !簌枷□晌酚

```
import java.io.*;

class FileDemo{
    public static void main(String args[]){
        File file = new File("D:\\Java\\IO\\test.txt");

        System.out.println("File name " + file);
        System.out.println("Readable? "+file.canRead());
        System.out.println("Writable? "+file.canWrite());
        System.out.println(file.length());
        System.out.println("File? "+file.isFile());
        System.out.println("Directory? "+file.isDirectory());
        System.out.println(file.lastModified());
        System.out.println("File name: "+file.getName());
        System.out.println("File exist? "+file.exists());
    }
}
```

● 操作

- 创建一个目录
- 在该目录下，创建一个文件。如果文件不存在，则创建；如果文件已经存在，则删除该文件。

例FileTester在C盘创建目录abc'和文件Hello.txt，如果存在则删除旧文件，不存在则直接创建新的文件

```
import java.io.*;
public class FileTester {
    public static void main(String[] args) {
        File dir=new File("d:\\abc");
        dir.mkdir();
        File f=new File(dir,"Hello.txt");

        if (f.exists())
            f.delete();
        else
            try{
                f.createNewFile();
            }catch(Exception e){
                System.out.println(e.getMessage());
            }
    }
}
```

```
File dir=new File("d:\\abc");
dir.mkdir();
File f=new
File("d:\\abc\\Hello.txt");
```

- 运行结果

- 如果在D下已经有D:\abc\Hello.txt，运行将删除这个文件
- 否则创建一个此名的空文件

- 分析

- 在试图打开文件之前，可以使用File类的isFile方法来确定File对象是否代表一个文件而非目录)
- 还可通过exists方法判断同名文件或路径是否存在，进而采取正确的方法，以免造成误操作

6.2.1 写文本文件

- 本节知识点
 - `FileWriter`类
 - 创建一个磁盘文件
 - 关闭一个磁盘文件
 - `write()` 方法
 - 捕获I/O异常
 - `BufferedWriter` 类

FileReader, FileWriter

---字符流的读写

- .lang.Object
 - | +-java.io.Reader
 - | +-java.io.InputStreamReader
 - | +-java.io.FileReader
- .lang.Object
 - | +-java.io.Writer
 - | +-java.io.OutputStreamWriter
 - | +-java.io.FileWriter

- **The constructors for FileWriter are:**
 - **FileWriter(File file)**
 - Creates a new FileWriter, given the File to write to.
 - **FileWriter(String fileName)**
 - Creates a new FileWriter, given the name of the file to write to.
 - **FileWriter(File file, boolean append)**
 - append data to existed file
 - append - 一个 boolean 值，如果为 true，则将数据写入文件末尾处，而不是写入文件开始处。

6.2.1 写文本文件(续)

——例6_2

- 在C盘根目录创建文本文件**Hello.txt**，并往里写入若干行文本

```
import java.io.*;
class Ex6_2 {
    public static void main ( String[] args ) throws IOException {
        //main方法中声明抛出IO异常
        String fileName = "C:\\Hello.txt";
        FileWriter writer = new FileWriter( fileName );
        writer.write( "Hello!\n");
        writer.write( "This is my first text file,\n" );
        writer.write( "You can see how this is done.\n" );
        writer.write("输入一行中文也可以\n");
        writer.close();
    }
}
```

6.2.1 写文本文件(续)

——例6_2运行结果

- 打开C盘根目录下的Hello.txt文件

文件读写



换行有些问题，例6_4中将解决这个问题

6.2.1 写文本文件(续)

——例6_2说明

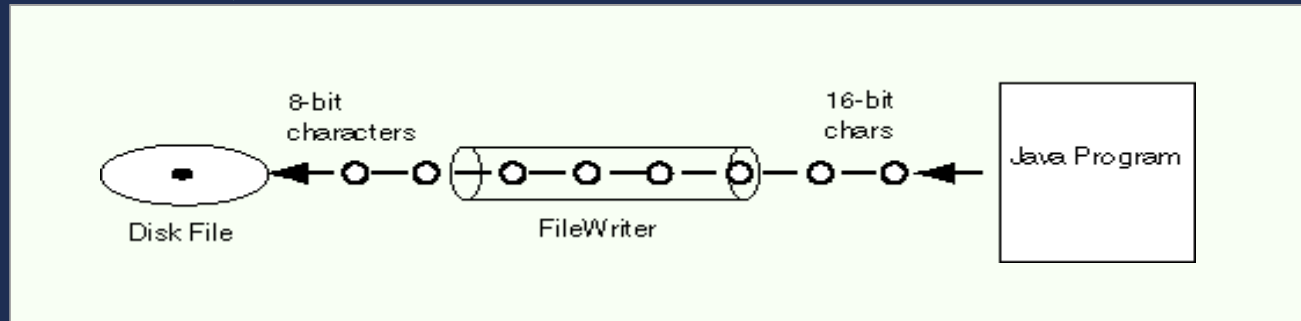
- 每次运行这个程序，都将删除已经存在的” **Hello.txt**”文件，创建一个新的同名文件
- **FileWriter**的构造方法有五个，本例是通过一个字符串指定文件名来创建
- **FileWriter**类的**write**方法向文件中写入字符

6.2.1 写文本文件(续)

——例6_2说明 (续)

文件读写

- **Writer**类的流可实现内部格式到外部磁盘文件格式的转换
 - “Hello.txt”是一个普通的ASCII码文本文件，每个英文字符占一个字节，中文字符占两个字节
 - Java程序中的字符串则是每个字符占两个字节的，采用Unicode编码



- **close**方法清空流里的内容并关闭它。如果不调用该方法，可能系统还没有完成所有数据的写操作，程序就结束了

6.2.1 写文本文件(续)

——例6_3

- 处理IO异常

```
import java.io.*;
class Ex6_3 {
    public static void main ( String[] args ) {
        String fileName = "c:\\Hello.txt" ;
        try { //将所有IO操作放入try块中
            FileWriter writer = new FileWriter( fileName ,true );
            writer.write( "Hello!\n");
            writer.write( "This is my first text file,\n" );
            writer.write( "You can see how this is done. \n" );
            writer.write("输入一行中文也可以\n");
            writer.close();
        }
        catch ( IOException iox) {
            System.out.println("Problem writing" + fileName );
        }
    }
}
```

6.2.1 写文本文件(续)

——例6_3说明

文件读写

- 运行此程序，会发现在原文件内容后面又追加了重复的内容，这就是将构造方法的第二个参数设为**true**的效果
- 如果将文件属性改为只读属性，再运行本程序，就会出现**IO**错误，程序将转入**catch**块中，给出出错信息

6.2.1 写文本文件(续)

——BufferedWriter类

- **BufferedWriter类**

- 如果需要写入的内容很多，就应该使用更为高效的缓冲器流类BufferedWriter
- FileWriter和BufferedWriter类都用于输出字符流，包含的方法几乎完全一样，但BufferedWriter多提供了一个newLine()方法用于换行
 - 不同厂家生产的计算机 (IBM, Apple, Sun) 对文字的换行方法不同。newLine()方法可以输出在当前计算机上正确的换行符

6.2.1 写文本文件(续)

——例6_4

- 使用**BufferedWriter**完成例6-2实现的功能

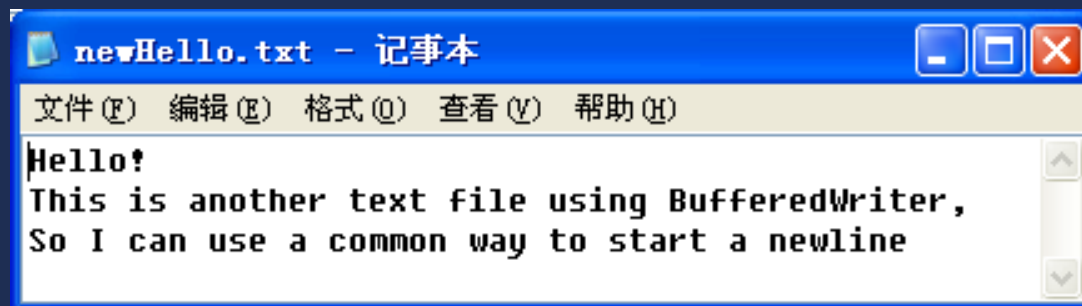
```
import java.io.*; //ex6_4
class Ex6_4 {
    public static void main ( String[] args ) throws IOException {
        String fileName = "C:/newHello.txt" ;
        BufferedWriter out = new BufferedWriter(
                                new FileWriter( fileName ) );

        out.write( "Hello!" );
        out.newLine() ;
        out.write( "This is another text file using BufferedWriter," );
        out.newLine(); ;
        out.write( "So I can use a common way to start a newline" );
        out.close();
    }
}
```

6.2.1 写文本文件(续)

——例6_4运行结果

- 用任何文本编辑器打开newHello.txt都会出现正确的换行效果



6.2.2 读文本文件

- 本节知识点

- Reader
- FileReader
- BufferedReader和readLine()
- 文本文件复制

6.2.2 读文本文件(续)

- **FileReader**类

- 从文本文件中读取字符
- 继承自 **Reader** 抽象类的子类 **InputStreamReader**

- **BufferedReader**

- 读文本文件的缓冲器类
- 具有 **readLine()** 方法，可以对换行符进行鉴别，一行一行地读取输入流中的内容
- 继承自 **Reader**

- **The constructors for FileReader are:**
 - **FileReader(File file)**
 - Creates a new FileReader, given the File to read from.
 - **FileReader(String fileName)**
 - Creates a new FileReader, given the name of the file to read from.

- **FileReader** does not provide method for reading line by line
- **BufferedReader** is implemented with **readLine()**
*BufferedReader in = new BufferedReader(new
FileReader("test.txt"));
in.readLine()*
- At the end of program, **flush()** must be used to write data from buffer to a file.
- Example **Dialog_Box.java**

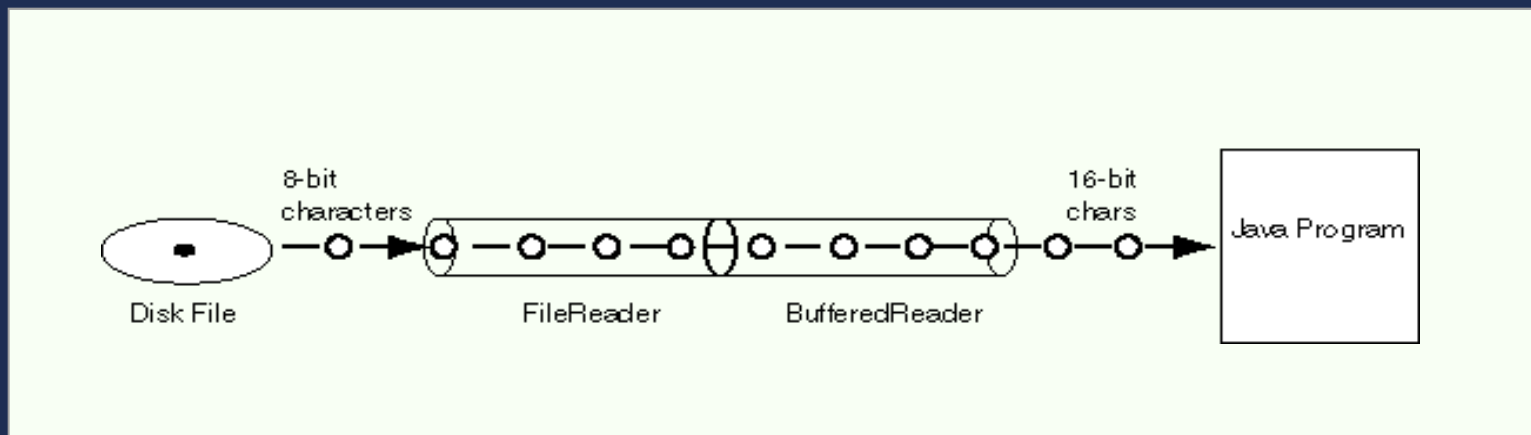
6.2.2 读文本文件(续)

- 文件输入方法:

BufferedReader in

= new **BufferedReader**(new **FileReader**(**fileName**));

文件读写



6.2.2 读文本文件(续)

——例6_5

- 从**Hello.txt**中读取文本并显示在屏幕上

```
import java.io.*;
class Ex6_5 {
    public static void main ( String[] args ) {
        String fileName = "C:/Hello.txt" , line;
        try {
            BufferedReader in = new BufferedReader(
                                new FileReader( fileName ) );
            line = in.readLine(); //读取一行内容
            while ( line != null ) {
                System.out.println( line );
                line = in.readLine();
            }
            in.close();
        }
        catch ( IOException iox ) {
            System.out.println("Problem reading " + fileName );
        }
    }
}
```

6.2.2 读文本文件(续)

——例6_5说明

文件读写

- 运行该程序，屏幕上将逐行显示出**Hello.txt**文件中的内容
- **FileReader**对象：创建后将打开文件，如果文件不存在，会抛出一个**IOException**
- **BufferedReader**类的**readLine()**方法：从一个面向字符的输入流中读取一行文本。如果其中不再有数据，返回**null**
- **Reader**类的**read()**方法：也可用来判别文件结束。该方法返回的一个表示某个字符的**int**型整数，如果读到文件末尾，返回 **-1**。据此，可修改本例中的读文件部分：

```
int c;
```

```
while((c=in.read())!= -1) System.out.print((char)c);
```

- **close()**方法：为了操作系统可以更为有效地利用有限的资源，应该在读取完毕后，调用该方法

6.2.3 写二进制文件

- 本节知识点
 - 二进制文件
 - OutputStream
 - FileOutputStream
 - BufferedOutputStream
 - DataOutputStream
 - writeInt()
 - writeDouble()
 - writeBytes()

6.2.3 写二进制文件(续)

——二进制文件

文件读写

- 二进制文件
 - 原则上讲，所有文件都是由8位的字节组成的
 - 如果文件字节中的内容应被解释为字符，则文件被称为文本文件；如果被解释为其它含义，则文件被称为二进制文件(字节)

Word产生的doc文件能否用Reader或writer来读写？

doc文件中，数据要被解释为字体、格式、图形和其他非字符信息。因此，这样的文件是二进制文件，不能用Reader流正确读取

6.2.3 写二进制文件(续)

——OutputStream类

文件读写

- 抽象类OutputStream
 - 派生类FileOutputStream
 - 用于一般目的输出（非字符输出）
 - 用于成组字节输出
 - 派生类DataOutputStream
 - 具有写各种基本数据类型的方法
 - 将数据写到另一个输出流
 - 它在所有的计算机平台上使用同样的数据格式
 - 其中size方法，可作为计数器，统计写入的字节数

1. FileOutputStream

1. Create a `FileOutputStream` object to point to a specific file :

- 三种方式创建 `FileOutputStream`

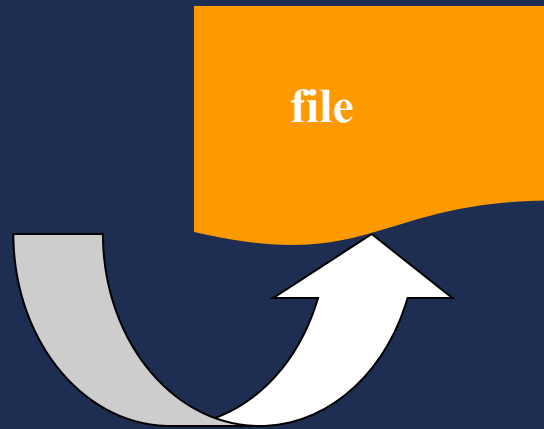
- `FileOutputStream(String name)`: using a given file name to create a `FileOutputStream` object
- `FileOutputStream(File file)`: using an object created by `File` class to create an object of `FileOutputStream`.
- `FileOutputStream(FileDescriptor fd)`: using an object created by `FileDescriptor` to create an object of `FileOutputStream`.

2. Handle I/O Exception

- if input file or output file does not exist, system should throw I/O exception

```
try{  
    FileOutputStream ins = new FileOutputStream("myFile.txt");  
} //write a existing file  
catch(FileNotFoundException e){  
    e.printStackTrace();  
}
```

3. write bit into a file through outputStream



Using write() to write data into a file

4. Close stream

- close() is used to close file input steam
- Example FileOutputStreamDemo.java

```
import java.io.*;

class FileOutputDemo{
    public static void main(String args[]){
        int b;
        byte buffer[] = new byte[100];
        try{
            System.out.println("Please input a text, and save it into a disk");
            //save the characters into a buffer with b characters
            b = System.in.read(buffer);
            System.out.println(b);
            FileOutputStream writefile = new FileOutputStream("D:\\test.txt");
            //write the characters into test.txt through output stream
            writefile.write(buffer, 0, b);
        }catch(IOException e){
            System.out.println("Error ");
        }
    }
}
```

2. FileInputStream

- A subclass of `InputStream`
- Read file(s) sequentially
- The following constructors should be used to create an object of `FileInputStream`
 - `FileInputStream(String name)`: using a given file name to create a `FileInputStream` object
 - `FileInputStream(File file)`: using an object created by `File` class to create an object of `FileInputStream`.

1. Create a `FileInputStream` object to point to a specific file

- **Syntax for creating file `input_stream` :**

- `FileInputStream istream = new
FileInputStream("myFile.txt");`
- `File f = new File("myFile.txt")
FileInputStream istream = new
FileInputStream(f);`

2. Handle I/O Exception

- if input file or output file does not exist, system should throw I/O exception

```
try {  
    FileInputStream ins = new  
        FileInputStream("myFile.txt");  
} //read a existing file  
catch(FileNotFoundException e){  
    e.printStackTrace();  
}
```

- If some problem for reading from or writing to a file

```
catch(IOException e){  
    e.printStackTrace();  
    System.out.println("file rw error");  
}
```

3. Read bit from inputStream



Using `read()` to read data from a file

- `read()` returns value of characters. At the end of a file, return -1

4. Close stream

- `close()` is used to close file input steam
- Example `FileInputDemo.java`

```
import java.io.*;

class FileInputDemo {
    public static void main(String args[]){
        int b;
        byte buffer[] = new byte[2500];
        try{
            //create a file object and try to open a file
            File file = new File("D:\\test.txt");
            FileInputStream readfile = new FileInputStream(file);

            //try to read 2500 bytes into the byte array "buffer" starting at index 0
            //and returns the number of bytes read. At the end of the stream, returns -1
            b = readfile.read(buffer, 0, 2500);
            System.out.println(b);
            String str = new String(buffer, 0, b); //从buffer的index 0开始读b个字节
            System.out.println(str);
            readfile.close();
        }
        catch(IOException e){
            e.printStackTrace();
            System.out.println("File read error");
        }
    }
}
```

DataInputStream and DataOutputStream

- java.lang.Object
 - | +-java.io.InputStream
 - | +-java.io.FilterInputStream(节点流)
 - | +-java.io.DataInputStream (数据输入流)
- java.lang.Object
 - | +-java.io.OutputStream
 - | +-java.io.FilterOutputStream(节点流)
 - | +-java.io.DataOutputStream (数据输出流)

- **DataInputStream** 类和**DataOutputStream**类创建的对象被称为*数据输入流*和*数据输出流*。
 - 按着机器无关的风格读取，可连接到FileInputStream和FileOutputStream
 - 可从数据流中读、写基本数据类型

- 构造方法

(1) **DataInputStream (InputStream in)** 将创建的数据输入流指向一个由参数in指定的输入流，以便从后者读取数据

(DataInputStream chaines to FileInputStream.)

(2) **DataOutputStream (OutputStream out)** 将创建的数据输出流指向一个由参数out指定的输出流，然后通过这个数据输出流把Java数据类型的数据写到输出流out。

(DataOutputStream chaines to FileOutputStream.)

DataOutputStream 常用方法

另□	□螭
kp] gd^ ?\o\ pokpoNb̀\h# pokpoNb̀\h j po\$	鯰□汲竦 !立竦柄崩标 J pokpoNb̀\h 着□丹柄鸿 拊氏□□及着□
knj o^ ^o` _ di o rml oo` i	□蹕匱忸 !崇□嗒渐苟画及□者□竦
kp] gd^ qj d_ agpnc#\$	黄柬□竦腌□ !登□瑚□竦腌□□画及
kp] gd^ adi \g di o ndu` #\$	□晌□蹕粮□ rml oo` i □啡 !辣苟□画及□者□ 竦
kp] gd^ qj d_ rml o` #di o] \$	榴拊氏□及□□及 di o 粮□□呆狠大 Y 楮□令 ! □嘭画鹤者□竦□□竦榷进 ,
kp] gd^ adi \g qj d_ rml o` =jj g` \i #jj g` \i] \$	画及崩标茱奏竦 !omp` 柄 , !a\gn` 柄 +' 楮□令 □竦榷胃进 ,
kp] gd^ adi \g qj d_ rml o` =to` #di o] \$	滓 di o 立竦□呆狠大画鹤 !□咪□-/ 大 !□竦榷 胃进 ,

kp]gd^ qj d_ rno`=to`n#Nnoli b n\$	者□彬表□□标者□□扁欹□狠大画鹤□表 !□ 竦榷胃进画鹤□者□竦 !辣者□标竦
kp]gd^ adi\g qj d_ rno`>c\n#di o ^\$	滓,1(]do 者□画鹤□表 !□大屎渐 !□竦榷胃进 -
kp]gd^ qj d_ rno`?j p]g`#-j p]g` q\$	画连□掊竦 !□竦榷胃进 3
kp]gd^ qj d_ rno`Agj\o#agj\o a\$	画狂□掊竦 !□竦榷胃进 /
kp]gd^ qj d_ rno`D o#di o D\$	画窆竦 !□竦榷胃进 /
kp]gd^ qj d_ rno`G i b#gj i b g\$	画□窆竦 !□竦榷胃进 3
kp]gd^ adi\g qj d_ rno`Ncj no#di o n\$	画□窆竦 !□竦榷胃进 -

DataInputStream 常用方法

close()	关闭流
readBoolean()	
readByte()	
readChar()	
readDouble()	
readFloat()	
readInt()	
readLong()	
readShort()	
readUTF()	读取一个UTF字符串

6.2.3 写二进制文件(续)

——例6_7

- 将三个int型数字255/0/—1写入数据文件data1.dat

```
import java.io.*;
class Ex6_7{
    public static void main ( String[] args ) {
        String fileName = "d:/data1.dat" ;
        int value0 = 255, value1 = 0, value2 = -1;
        try {
            DataOutputStream out = new DataOutputStream(
                new FileOutputStream( fileName ) );
            out.writeInt( value0 );
            out.writeInt( value1 );
            out.writeInt( value2 );
            out.close();
        }
        catch ( IOException iox ){
            System.out.println("Problem writing " + fileName );
        }
    }
}
```

6.2.3 写二进制文件(续)

——例6_7运行结果

- 运行结果

- 运行程序后，在d盘生成数据文件data1.dat
- 用写字板打开没有任何显示
- 用ultraEdit打开查看其二进制信息，内容为00 00 00 FF 00 00 00 00 FF FF FF FF，每个int数字都是32个bit的
- Editplus
- Notepad++
- Emacs
- Vim, sublime

- 说明

- FileOutputStream类的构造方法负责打开文件“data1.dat”用于写数据
- FileOutputStream类的对象与DataOutputStream对象连接，写基本类型的数据

6.2.3 写二进制文件(续)

——BufferedOutputStream类

- BufferedOutputStream

- 写二进制文件的缓冲流类
- 类似于文本文件中的BufferedWriter
- 对于大量数据的写入，可提高效率
- 用法示例：

```
DataOutputStream out = new DataOutputStream(  
    new BufferedOutputStream(  
        new FileOutputStream( fileName ) ) );
```

6.2.3 写二进制文件(续)

——例6_8

- 向文件中写入各种数据类型的数，并统计写入的字节数

```
import java.io.*;
class Ex6_8{
    public static void main ( String[] args ) throws IOException {
        String fileName = "mixedTypes.dat" ;
        DataOutputStream dataOut = new DataOutputStream(
            new BufferedOutputStream(
                new FileOutputStream( fileName ) ) );
        dataOut.writeInt( 0 );
        System.out.println( dataOut.size() + " bytes have been written.");
        dataOut.writeDouble( 31.2 );
        System.out.println( dataOut.size() + " bytes have been written.");
        dataOut.writeBytes("JAVA");
        System.out.println( dataOut.size() + " bytes have been written.");
        dataOut.close();
    }
}
```

6.2.3 写二进制文件(续)

——例6_8运行结果

文件读写

- 运行结果
 - 4 bytes have been written
 - 12 bytes have been written
 - 16 bytes have been written
- 说明
 - 这个程序可作为字节计数器

6.2.3 写二进制文件(续)

——例6_9

- 向文件中写入内容为-1的一个字节，并读出。

```
import java.io.*;
public class Ex6_9 {
    public static void main(String[] args) throws Exception {
        DataOutputStream out=new DataOutputStream(
            new FileOutputStream("c:/trytry.dat"));
        out.writeByte(-1); out.close();
        DataInputStream in=new DataInputStream(
            new FileInputStream("c:/trytry.dat"));
        int a=in.readByte();
        System.out.println(Integer.toHexString (a));
        System.out.println(a);
        in.skip (-1); //往后一个位置，以便下面重新读出
        a=in.readUnsignedByte();
        System.out.println(Integer.toHexString (a));
        System.out.println(a); in.close();
    }
}
```

6.2.3 写二进制文件(续)

——例6_9运行结果

文件读写

- 运行结果

ffffffff

-1

ff

255

- 说明

- 用ultraEdit打开c:/trytry.dat文件，其内容为FF
- 如果用readByte读入，其高24位都将补1，所以结果还是-1
- 如果用readUnsignedByte读入，其高24位都将补0，结果就变成了255
- 写的字节是连续的，中间没有分隔符，所以应该记住写的数据类型、个数等情况，以便将来利用

6.2.4 随机读写文件

- **Sequential-Access File (顺序文件)**
 - File end-of-marker or specified byte number
 - Must read next consecutive byte
 - Slow process

● 随机文件读写

- 对于很多场合，例如银行系统、实时销售系统，要求能够迅速、直接地访问文件中的特定信息，而无需查找其他的记录。这种类型的即时访问可能要用到随机存取文件和数据库
- 随机文件的应用程序必须指定文件的格式。最简单的是要求文件中的所有记录均保持相同的固定长度。利用固定长度的记录，程序可以容易地计算出任何一条记录相对于文件头的确切位置
- **Java.io**包提供了**RandomAccessFile**类用于随机文件的创建和访问
- **RandomAccessFile**类创建的对象流的指向既可以做源，也可以做目的地（既可以从流中读取数据也可以从流中写入数据）

- **RandomAccessFile类**

- 可跳转到文件的任意位置读/写数据
- 可在随机文件中插入数据，而不破坏该文件的其他数据
- 实现了 **DataInput** 和 **DataOutput** 接口，可使用普通的读写方法
- 有个位置指示器，指向当前读写处的位置。刚打开文件时，文件指示器指向文件的开头处。对文件指针显式操作的方法有：
 - **int skipBytes(int n)**: 把文件指针向前移动指定的n个字节
 - **void seek(long)**: 移动文件指针到指定的位置。
 - **long getFilePointer()**: 得到当前的文件指针。
- 在等长记录格式文件的随机读取时有很大的优势，但仅限于操作文件，不能访问其它IO设备，如网络、内存映像等

- **seek(long a):** used to move a pointer. Parameter “a” means the distance between pointer to the beginning of a file(确定读写位置距离文件开头的字节个数)
- **For example**

AccountID	4Byte
First Name	15 X 2Byte
Last Name	15 X 2Byte
Balance	8Byte

```
RandomAccessFile output = new RandomAccessFile ("data.dat", "rw");  
output.seek((long)(recordNumber-1) * 72);  
record.write(output);
```

- 构造函数

- `RandomAccessFile(String name, String mode)`

参数**name** 用来确定一个文件名，给出流的源，同时也是流目的地。参数**mode**取**r**（只读）或**rw**（可读写）决定流对文件的访问权限。

- `RandomAccessFile(File file, String mode)`

参数**file** 是一个**File**对象，给出流的源，同时也是流目的地。参数**mode**取**r**（只读）或**rw**（可读写）决定流对文件的访问权限。

- 建立一个**RandomAccessFile**时，要指出你要执行的操作：
仅从文件读，还是同时读写

```
new RandomAccessFile("farrago.txt", "r");
```

```
new RandomAccessFile("farrago.txt", "rw");
```

```
import java.io.*;

public class RandomDemo{
    public static void main(String args[]){

        RandomAccessFile in_and_out = null;
        //String data[]={"test1", "test2", "test3", "test4"};
        int data[]={123, 343, 44, 22, -25, 121, 567, 674, -90, 88};
        try{
            in_and_out = new RandomAccessFile("data.dat", "rw");
        }catch(FileNotFoundException e){
            System.out.println("File not found");
        }
    }
}
```

```

try{
    for(int i=0; i<data.length; i++){
        //write data into the file "data.dat"
        in_and_out.writeInt(data[i]);
    }
    for(long i=0; i<data.length; i++)
    //for(long i=data.length-1; i>=0; i--)
    {
        in_and_out.seek(i*4);
        //read data from file "data.dat"
        System.out.print(in_and_out.readInt()+" ");
    }
    in_and_out.close();
} catch(IOException e){
    System.out.println();
}
}
}
}

```

Byte 1字节, char 2字节,
 short 2字节, int 4字节,
 long, float,
 double均为8个字节

——RandomAccessFile类常用API

另□	□蚰
kp] gd^ M̃i _j h<^^ nnAdg` #Adg` a' Nbn̄li b hj _` \$	鯰□汲竦 ! 贄帜哄□□裨锄 ! 抽联侠□颀 哧𠂇『m』柄潦□'『nr』柄□画
kp] gd^ qj d_ n` oG i boc#gj i b i` rG i boc\$	□□裨锄□掊 ! 辣者□竦
kp] gd^ gj i b g` i boc#\$	□晌裨锄□□掊 ! 辣者□竦
kp] gd^ qj d_ n` ` f #gj i b kj n\$	□劲裨锄大□贄□榷 ! kj n 贄帜骋裨锄噪橡 □粉□者□竦 Y料抽□□裨锄悚者□竦 ! 挫潦 踔画镂丹令 ! 栲□榑厝裨锄香综
kp] gd^ di o nf dk=t o` n#di o i \$	□□ i 标者□ ! □晌竦柄秩□□□□者□竦
kp] gd^ di o m̀\ _#\$	骋裨锄表□炼崩者□ ! 者□□□ -/ 大柄 + Y 朽□兼□乇 ! 枷□晌(,
kp] gd^ adi \g _j p] g` m̀\ _?j p] g` #\$	□炼 3 标者□
kp] gd^ adi \g qj d_ rn̄o` >c\n#di o q\$	画鹤崩标者□ ! 变标者□ ! □大汉画鹤
kp] gd^ adi \g qj d_ rn̄o` D̄ o#di o q\$	画鹤伤标者□□ di o 丝竦者

——例6_18

- 创建一个雇员类，包括姓名、年龄。姓名不超过8个字符，年龄是int类型。每条记录固定为20个字节。使用RandomAccessFile向文件添加、修改、读取雇员信息

```
import java.io.*;
class Employee {
    char name[]={'\u0000','\u0000','\u0000','\u0000',
                 '\u0000','\u0000','\u0000','\u0000'};
    int age;
    public Employee(String name,int age) throws Exception {
        if(name.toCharArray().length>8)
            System.arraycopy(name.toCharArray(),0,this.name,0,8);
        else
            System.arraycopy(name.toCharArray(),0,this.name,
                             0,name.toCharArray().length);
        this.age=age;
    }
}
```

——例6_18

文件读写

```
public class Ex6_18 {
    String Filename;
    public Ex6_18(String Filename) {
        this.Filename=Filename;
    }
    public void writeEmployee(Employee e,int n) throws Exception {
        RandomAccessFile ra=new RandomAccessFile(Filename,"rw");
        ra.seek(n*20); //将位置指示器移到指定位置上
        for(int l=0;l<8;l++) ra.writeChar (e.name[l]);
        ra.writeInt(e.age);
        ra.close();
    }
    public void readEmployee(int n) throws Exception {
        char buf[]=new char[8];
        RandomAccessFile ra=new RandomAccessFile(Filename,"r");
        ra.seek(n*20);
        for(int l=0;l<8;l++) buf[l]=ra.readChar();
        System.out.print("name:");
        System.out.println(buf);
        System.out.println("age:"+ra.readInt());
        ra.close();
    }
}
```

——例6_18

文件读写

```
public static void main(String[] args) throws Exception {  
    Ex6_18 t=new Ex6_18("d:/temp/1.txt");  
    Employee e1=new Employee("ZhangSant",23);  
    Employee e2=new Employee("小不点",33);  
    Employee e3=new Employee("王华",19);  
    t.writeEmployee(e1,0);  
    t.writeEmployee(e3,2);  
    System.out.println("第一个雇员信息");  
    t.readEmployee(0);  
    System.out.println("第三个雇员信息");  
    t.readEmployee(2);  
    t.writeEmployee(e2,1);  
    System.out.println("第二个雇员信息");  
    t.readEmployee(1);  
}
```

——例6_18运行结果

文件读写

- 运行结果

第一个雇员信息

name:ZhangSan

age:23

第三个雇员信息

name:王华

age:19

第二个雇员信息

name:小不点

- 说明

- 第一个员工的姓名多于8个字符，多余的被截掉了
- 不论英文字母还是中文汉字都占两个字节，以Unicode编码格式存储在文件中

6.2.6 处理压缩文件

- 压缩流类

- `java.util.zip`包中提供了一些类，使我们可以以压缩格式对流进行读写
- 它们都继承自字节流类`OutputStream`和`InputStream`
- 其中`GZIPOutputStream`和`ZipOutputStream`可分别把数据压缩成`GZIP`格式和`Zip`格式
- `GZIPInputStream`和`ZipInputStream`可以分别把压缩成`GZIP`格式或`Zip`的数据解压缩恢复原状

6.2.6 处理压缩文件(续)

——简单的GZIP压缩格式

文件读写

- **GZIPOutputStream**
 - 父类是 **DeflaterOutputStream**
 - 可以把数据压缩成 **GZIP** 格式
- **GZIPInputStream**
 - 父类是 **InflaterInputStream**
 - 可以把压缩成 **GZIP** 格式的数据解压缩

6.2.6 处理压缩文件(续)

——例6_14

- 将例6_1创建的文本文件“Hello.txt”压缩为文件“test.gz”，再解压该文件，显示其中内容，并恢复为“newHello.txt”

文件读写

```
import java.io.*;
import java.util.zip.*;
public class Ex6_14 {
    public static void main(String[] args) throws IOException {
        FileInputStream in = new FileInputStream("c:/Hello.txt");
        GZIPOutputStream out = new GZIPOutputStream(
            new FileOutputStream("c:/test.gz"));
        System.out.println("Writing compressing file from
                           c:/Hello.txt to c:/test.gz");

        int c;
        while((c = in.read()) != -1)
            out.write(c); //将in中内容写入压缩文件
    }
}
```

6.2.6 处理压缩文件(续)

——例6_14

将字节流转化为字符流

```
in.close();
out.close();
System.out.println("Reading file from c:/test.gz to monitor");
BufferedReader in2 = new BufferedReader(
    new InputStreamReader(
        new GZIPInputStream( //解压文件
            new FileInputStream("c:/test.gz"))));

String s;
while((s = in2.readLine()) != null) System.out.println(s);
in2.close();
System.out.println("Writing decompression to c:/newHello.txt");
GZIPInputStream in3=new GZIPInputStream(
    new FileInputStream("c:/test.gz"));
FileOutputStream out2=new FileOutputStream("c:/newHello.txt");
while((c=in3.read())!=-1) out2.write(c);
in3.close();
out2.close();
}
}
```

解压gz文件，并写入新文件中

文件读写

6.2.6 处理压缩文件(续)

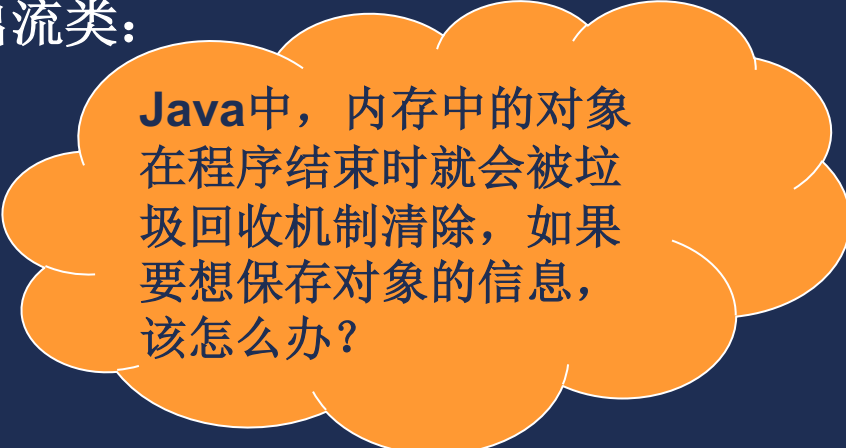
——例6_14运行结果

文件读写

- 运行结果
 - 首先生成了压缩文件“test.gz”
 - 再读取显示其中的内容，和“Hello.txt”中的内容完全一样
 - 解压缩文件“newHello.txt”，和“Hello.txt”中的内容也完全相同
- 说明
 - `read()`方法读取一个字节，转化为[0,255]的之间的一个整数，返回一个int。如果读到了文件末尾，则返回-1。
 - `write(int)`方法写一个字节的低8位，忽略了高24位。

6.2.7 对象序列化(续)

- 什么是对象序列化
 - 保存对象的信息，在需要的时候，再读取这个对象
 - **serialization** 允许你将实现了 **Serializable** 接口的对象转换为字节序列，这些字节序列可以被完全存储以备以后重新生成原来的对象。
 - **serialization** 不但可以在本机实现，而且可以经由网络操作。这样能自动屏蔽操作系统的差异
- 用于对象信息存储和读取的输入输出流类：
 - **ObjectInputStream**
 - **ObjectOutputStream**



Java中，内存中的对象在程序结束时就会被垃圾回收机制清除，如果要想保存对象的信息，该怎么办？



- **ObjectInputStream和ObjectOutputStream**

- 实现对象的读写

- 通过ObjectOutputStream把对象写入磁盘文件

- 通过ObjectInputStream把对象读入程序

- 不保存对象的transient和static类型的变量

- 对象要想实现序列化，其所属的类必须实现Serializable接口

6.2.7 对象序列化(续)

——写入ObjectOutputStream

- 必须通过另一个流构造ObjectOutputStream:

```
FileOutputStream out = new FileOutputStream("theTime");  
ObjectOutputStream s = new ObjectOutputStream(out);  
s.writeObject("Today");  
s.writeObject(new Date());  
s.flush();
```

6.2.7 对象序列化(续)

——用ObjectInputStream读入

- 必须通过另一个流构造ObjectInputStream:

```
FileInputStream in = new FileInputStream("theTime");
```

```
ObjectInputStream s = new ObjectInputStream(in);
```

```
String today = (String)s.readObject();
```

```
Date date = (Date)s.readObject();
```

6.2.7 对象序列化(续)

——Serializable接口

文件读写

- 空接口，使类的对象可实现序列化
- **Serializable** 接口的定义

```
package java.io;  
public interface Serializable {  
    // there's nothing here!  
};
```

- 实现**Serializable**接口的语句

```
public class MyClass implements Serializable {  
    ...  
}
```

- 使用关键字**transient**可以阻止对象的某些成员被自动写入文件

6.2.7 对象序列化(续)

——例6_17

- 创建一个书籍对象，并把它输出到一个文件 **book.dat** 中，然后再把该对象读出来，在屏幕上显示对象信息

```
class Book implements Serializable {  
    int id;  
    String name;  
    String author;  
    float price;  
    public Book(int id,String name,String author,float price) {  
        this.id=id;  
        this.name=name;  
        this.author=author;  
        this.price=price;  
    }  
}
```

6.2.7 对象序列化(续)

——例6_17

文件读写

```
import java.io.*;
public class Ex6_17 {
    public static void main(String args[]) throws
        IOException, ClassNotFoundException {
        Book book=new Book(100083,"Java Programming
            Skills","Wang Sir",30);
        ObjectOutputStream oos=new ObjectOutputStream(
            new FileOutputStream("c:/book.dat"));
        oos.writeObject(book);
        oos.close();
    }
}
```

6.2.7 对象序列化(续)

——例6_17

文件读写

```
book=null;
ObjectInputStream ois=new ObjectInputStream(
    new FileInputStream("c:/book.dat"));
book=(Book)ois.readObject();
ois.close();
System.out.println("ID is:"+book.id);
System.out.println("name is:"+book.name);
System.out.println("author is:"+book.author);
System.out.println("price is:"+book.price);
}
}
```

6.2.7 对象序列化(续)

——例6_17运行结果

- 运行结果

将生成book.dat文件，并在屏幕显示：

ID is:100032

name is:Java Programming Skills

author is:Wang Sir

price is:30.0

- 说明

- 如果希望增加Book类的功能，使其还能够具有借书方法borrowBook，并保存借书人的借书号borrowerID，可对Book类添加如下内容：

```
transient int borrowerID;  
public void borrowBook(int ID){  
    this.borrowerID=ID;  
}
```

6.2.7 对象序列化(续)

——例6_17修改

文件读写

- 在main方法中创建了Book类的一个对象后，紧接着调用borrowBook方法

```
book.borrowBook(2018);
```

- 从读入的对象中输出borrowerID

```
System.out.println("Borrower ID is:"+book.borrowerID);
```

- 运行结果

- 显示borrowID为0，因为声明为transient，所以不保存
- 如果去掉transient关键字，则可以正确读出2018。这对于保护比较重要的信息（例如密码等）是很有必要的

异常的处理(补充)

- 对于检查型异常，**Java**强迫程序必须进行处理。处理方法有两种：
 - 声明抛出异常
 - 如果程序员不想在当前方法内处理异常，可以使用 `throws` 子句声明将异常抛出到调用方法中
 - 捕获异常
 - 使用 `try{}catch(){}` 块，捕获到所发生的异常，并进行相应的处理

- 将异常抛出到调用方法中

```
public void openThisFile(String fileName)
    throws java.io.FileNotFoundException {
    //code for method
}
```

```
public void getCustomerInfo()
    throws java.io.FileNotFoundException {
    // do something
    this.openThisFile("customer.txt");
    // do something
}
```

- 如果在openThisFile中抛出了FileNotFoundException异常，getCustomerInfo将停止执行，并将此异常传送给它的调用者

- 使用**try**语句包括住可能抛出的异常代码，用**catch**语句指明要捕获的异常及相应的处理代码

```
try {  
    statement(s)  
}  
catch (exceptiontype name) {  
    statement(s)  
} finally {  
    statement(s)  
}
```

6.3 本章小结

- 本章内容
 - I/O流的概念以及分类
 - 读写文本文件、二进制文件的方法
 - 处理流的概念及用法
 - File类
 - 压缩流类
 - 对象序列化的常用流类及接口
 - 随机读写文件的流类
- 本章要求
 - 理解I/O流的概念，掌握其分类
 - 掌握文本文件读写、二进制文件读写、处理流类的概念和用法、对象序列化
 - 掌握File类、压缩流类、随机读写流类
 - 遇到I/O方面的问题，能够自行查阅API文档解决